

○ 2024 FA [1] Software Development Practicum ET 721 BT [32862] [Lecture]  
[Queensborough CC]

## **Django Lab project**

### ***Create a Simple Blog App with Django***

#### **Objective**

Build a basic blog application in Django that allows users to view, create, and edit blog posts. This project will help you understand Django's core concepts, such as models, views, templates, forms, and URL routing.

#### **Project Requirements**

This project should cover the following core features of Django:

##### **Part 1: Setting up the Project and App**

1. Create a GitHub repository as 'ET721\_lab\_blog'. After it, open the repository in codespace.
2. In codespace:
  - Install the python virtual environment.
  - Connect to the virtual environment and run the Django app to test.
3. **Set Up Django Project**
  - Create a new Django project named blogproject.
  - Inside this project, create a Django app called blog.
4. **Configure Settings**
  - Add the blog app to the INSTALLED\_APPS list in your project's settings file.

##### **Part 2: Define the Model**

In this part, you will define a single model to represent blog posts.

##### **1. BlogPost Model**

In blog/models.py, create a model for blog posts with the following fields:

- title: CharField with a maximum length of 200 characters.
- content: TextField to store the content of the blog post.
- created\_at: DateTimeField with auto\_now\_add=True to record the date and time when the post was created.
- updated\_at: DateTimeField with auto\_now=True to record the date and time when the post was last updated.

## 2. Run Migrations

- Run makemigrations and migrate to create the database table for your model.

```
python manage.py makemigrations
python manage.py migrate
```

## Part 3: Create Views and URL Patterns

### 1. Views for Basic Blog Functionality

Create views in `blog/views.py` for the following functionalities:

- **List View:** Display a list of all blog posts.
- **Detail View:** Display a single blog post in detail.
- **Create View:** Allow users to add a new blog post.
- **Update View:** Allow users to edit an existing blog post.

### 2. URL Patterns

Define URL patterns in `blog/urls.py` to route requests to the appropriate views. URL patterns should include:

- `/` for the list view (e.g., home).
- `/post/<id>/` for the detail view of a specific post.
- `/post/new/` for the view to create a new post.
- `/post/<id>/edit/` for the view to edit a post.

Example `urls.py`:

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path("", views.post_list, name='post_list'),
    path('post/<int:id>/', views.post_detail, name='post_detail'),
    path('post/new/', views.post_create, name='post_create'),
    path('post/<int:id>/edit/', views.post_edit, name='post_edit'),
]
```

Include `blog/urls.py` in the main `urls.py` file of your Django project.

## Part 4: Create Templates

### 1. Templates for Each View

Create an HTML file for each view in the `templates/blog` directory:

- Displays a list of all blog posts with their titles and creation dates.
- Each post title should link to the detail view of the post.
- Displays the post's title, content, and creation date.
- Include an "Edit" link to update the post.
- Display a form to add or edit a post with fields for title and content.

- Use Django's form syntax (`{{ form.as_p }}`) to render the form fields.
- Include a "Save" button to submit the form.
  - **List View Template (post\_list.html):**
  - **Detail View Template (post\_detail.html):**
  - **Create and Edit Templates (post\_form.html):**

## 2. Template Base File

Create a `base.html` template to serve as the base for all other templates. It should contain a basic HTML structure with a header and navigation links to "Home" and "New Post".

## Part 5: Create Forms

### 1. Django Forms for BlogPost Model

Create a form in `blog/forms.py` using Django's `ModelForm` to create or edit a blog post.

```
from django import forms
from .models import BlogPost

class BlogPostForm(forms.ModelForm):
    class Meta:
        model = BlogPost
        fields = ['title', 'content']
```

## Part 6: Add Basic Styling

Add CSS styling to make the blog application look more polished. You can place your CSS in a file under `static/blog/styles.css` and link it in `base.html`.

## Part 7: Testing Your App

### 1. Run the Server

Start the development server and ensure that each feature of your blog app is working.

```
python manage.py runserver
```

## Submission Requirements

### 1. Code Structure

- Ensure code is organized into models, views, templates, and forms.
- Keep code clean and follow Django best practices.

### 2. Documentation

- Include a `README.md` file that describes the project and provides instructions on how to set up and run the app.

### 3. Publish Django

- Publish your Django project through Heroku or `pythonanywhere`. Will be shown in during class lecture

## Evaluation Criteria

1. **Completeness:** Does the project include all required views, templates, and functionality?

2. **Code Quality:** Is the code well-organized and readable?
3. **UI and Design:** Is the application user-friendly and visually clean?
4. **Optional Features:** Extra credit for basic styling and test cases.

### Lab schedule

1. Complete Part 1 and 2 during the class on Monday, November 11.
2. Complete Part 3 and 4 during the class on Friday, November 15.
3. Complete Part 5, 6, and 7, by Sunday, November 17.

### Grading:

- 15 points. Grade Part 1 and 2 by the end of class on Monday, November 11.
- 15 points. Grade Part 3 and 4 by the end of class on Friday, November 15.
- 40 points. Publish completed app by Sunday, November 17.
- TOTAL POINTS: 70 points.



### Django project - Create a Simple Blog App with Django



Due November 17 at 11:59 PM



Ends Nov 17, 2024 11:59 PM

---